



Opciones creación de páginas web

Laboratorio de modelación matemática

Repositorio para artículos, tesis, documentos importantes, etc.

fulano@buapm.mx

Contenido (1/2)

I. Introducción

- Motivación

2. Lenguaje común: conceptos básicos

- ¿Qué es Markdown?
- ¿Qué es Git y GitHub?
- Terminal y shell

3. Infraestructura web mínima

- Host y web
- HTML, CSS, JavaScript

4. Opciones para el sitio del laboratorio

- GitHub Pages + Jekyll
- Manejo de archivos pesados
- WordPress
- Wix / No-code
- Opción D: GitLab Pages + CI/CD
- Opción E: Hosting institucional / servidor propio
- Opción F: Repositorios científicos + sitio mínimo

Contenido (2/2)

5. Visión global

6. Comparación técnica

7. Prueba de concepto

- Lecciones del piloto

8. Resultado real: prueba piloto

- Repositorio funcional

9. Caso real: piloto (MVP)

- Sitio en beta: 9th_semester
- Cómo se construyó el MVP

10. Conclusiones

11. Referencias

¿Por qué un sitio web del laboratorio?

Motivación

En un laboratorio, el conocimiento no solo se produce:

- se documenta,
- se comparte,
- se preserva.

Objetivo de esta presentación

Comparar opciones para crear un **repositorio + sitio web** del laboratorio (artículos, tesis, reportes, código y recursos), priorizando:

- sostenibilidad económica,
- compatibilidad científica (LaTeX, BibTeX, PDFs, MATLAB, MATHEMATICA, Python, software de externos, etc),
- facilidad de uso y mantenimiento.

Markdown

lenguaje de marcado ligero

- **Markdown** es un formato de texto simple para escribir contenido estructurado.
- Se convierte fácilmente a **HTML/PDF** y se usa mucho en documentación científica.

Ejemplo mínimo

Título ****negritas**** - listas [enlaces](...)

¿Qué permite hacer?

Porque permite escribir **contenido académico** sin depender de editores propietarios y se integra perfecto con repositorios y sitios web.

Git

Sistema de control de versiones distribuido

- **Git** es un sistema de control de versiones **local y distribuido**.
- Permite registrar el historial de cambios de un proyecto:
 - documentos,
 - código,
 - datos y resultados.
- Facilita volver a versiones anteriores y trabajar sin perder información.

Creador

Git fue creado en 2005 por **Linus Torvalds**, el creador del kernel de **Linux**, como respuesta a la necesidad de un sistema rápido, robusto y confiable para el desarrollo colaborativo del sistema operativo.

GitHub

Plataforma de colaboración y alojamiento de repositorios

- **GitHub** es una plataforma en la nube que aloja repositorios Git.
- Fue fundada en 2008 con el objetivo de facilitar el trabajo colaborativo basado en Git.
- Permite compartir proyectos, revisar cambios y mantener repositorios públicos o privados.

Interfaz y uso

- **Git** se utiliza principalmente mediante una **interfaz de línea de comandos (CLI)**.
- **GitHub** proporciona una **interfaz gráfica (GUI)** accesible desde un navegador web.
- Ambas herramientas se complementan dentro de un mismo flujo de trabajo.

Terminal y shell

Cómo se comunican el usuario y el sistema

Una **terminal** es un programa que permite al usuario introducir comandos en forma de texto.

La **shell** (bash, zsh, fish, etc) es el intérprete que:

- recibe los comandos escritos en la terminal,
- los interpreta,
- y se comunica con el **kernel** del sistema operativo para ejecutar las acciones solicitadas.

La terminal es la interfaz visible; la shell es quien realmente habla con el sistema operativo.

Git desde la línea de comandos

Flujo de trabajo con bash

- **Git** se ejecuta como un programa dentro de la shell.
- El usuario interactúa mediante comandos escritos.
- Este enfoque es:
 - preciso,
 - reproducible,
 - usado en ciencia y software.

Comandos típicos

```
git status
git add .
git commit -m "mensaje"
git push
```



```
git status
git add .
git commit -m "mensaje"
git push
```

The screenshot shows a terminal window with a dark background. It displays the output of several Git commands. The first command is 'git status', which shows the current state of the working directory. The second command is 'git add .', which adds all files in the current directory to the staging area. The third command is 'git commit -m "mensaje"', which commits the staged files with the message 'mensaje'. The fourth command is 'git push', which pushes the committed changes to the remote repository. The output of each command is shown below the command itself.

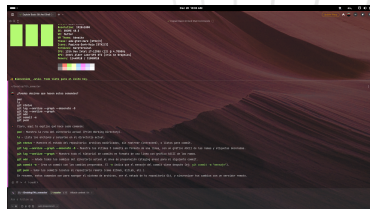
Ejemplo ilustrativo de Git usado desde
bash

Terminal moderna: WARP

Interfaz gráfica y asistencia con IA

- **WARP** es una terminal moderna con interfaz gráfica (GUI).
- Incorpora:
 - autocompletado inteligente,
 - historial visual,
 - sugerencias de comandos,
 - asistencia mediante IA.
- Reduce la barrera de entrada al uso de Git y la terminal.

No reemplaza a Git ni a la shell, sino que **mejora la interacción** con ellas.



Ejemplo ilustrativo de WARP con sugerencias asistidas

¿Qué es un host y qué es la web?

Un **host** es un servicio/servidor que almacena y entrega tu sitio para que cualquiera lo visite.

Dos ideas clave

- Un sitio web es un conjunto de archivos accesibles por URL.
- Esos archivos se “sirven” desde un host (servidor).

Objetivo institucional

Que el laboratorio tenga una presencia estable y un repositorio accesible, sin depender de una computadora personal.

HTML, CSS y JavaScript

desarrollo web

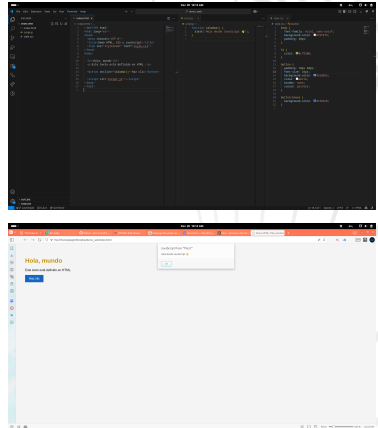
- **HTML:** estructura del contenido (títulos, párrafos, enlaces).
- **CSS:** apariencia visual (colores, tipografía, diseño).
- **JavaScript:** interacción y comportamiento (botones, eventos, respuestas dinámicas).

Analogía rápida

HTML = esqueleto

CSS = piel / estilo

JavaScript = comportamiento



Código (arriba) y resultado en navegador (abajo)

¿Qué es GitHub Pages?

Servicio de alojamiento web de GitHub

- **GitHub Pages** es un servicio de **hosting gratuito** proporcionado por GitHub.
- Permite publicar **sitios web estáticos** directamente desde un repositorio Git.
- Cada cambio en el repositorio puede reflejarse automáticamente en el sitio web.

GitHub Pages convierte un repositorio en un **sitio web público, versionado y reproducible**.

Uso típico en ciencia

Páginas personales de investigadores, documentación de proyectos, laboratorios académicos y repositorios de resultados.

¿Qué es Jekyll?

Generador de sitios estáticos

- **Jekyll** es un **generador de sitios estáticos**.
- Convierte archivos escritos en **Markdown** en páginas web HTML.
- Organiza automáticamente:
 - menús,
 - secciones,
 - índices,
 - enlaces entre documentos.

Relación con GitHub Pages

GitHub Pages puede ejecutar Jekyll de forma automática cada vez que el repositorio se actualiza.

Ventaja científica

Permite escribir contenido académico en texto plano, sin depender de editores propietarios.

Opción A: GitHub Pages + Jekyll

Propuesta principal

- Hosting gratuito para sitios estáticos.
- Generadores como **Jekyll** convierten **Markdown** a páginas web.
- Ideal para documentación científica, PDFs, ecuaciones.

Ventajas

- Costo **\$0** (dominio opcional)
- Seguridad alta (sin base de datos)
- Versionado y trazabilidad (Git)

Archivos pesados de laboratorio

Datos, PDFs grandes y reproducibilidad

Si el laboratorio genera grandes volúmenes de datos o archivos que superan los límites habituales de un repositorio, existen alternativas:

- **Git LFS (Large File Storage):**
 - Extensión de Git para manejar archivos grandes.
 - GitHub ofrece en la cuenta gratuita:
 - **1 GB** de almacenamiento LFS
 - **1 GB** de ancho de banda mensual
- **Almacenamiento externo especializado:**
 - Para datasets, videos experimentales o resultados muy pesados.
 - Servicios recomendados: **OSF (Open Science Framework)**, **Zenodo** o **Google Drive**.
 - Los archivos se enlazan desde el sitio Jekyll.

Integración de datos externos

Zenodo, OSF y repositorios científicos

GitHub Pages y Jekyll permiten **integrar enlaces directos** a repositorios externos que almacenan datos pesados del laboratorio.

- El sitio web funciona como un **índice centralizado** del proyecto.
- Los datasets y archivos grandes se alojan en plataformas especializadas.
- Desde cada página se puede redirigir al recurso externo con un solo clic.

Ejemplo en Markdown (GitHub Pages / Jekyll)

```
# Dataset del experimento EEG
```

Los datos completos se encuentran disponibles en:

```
[Zenodo - DOI: 10.5281/zenodo.1234567]
```

```
(https://doi.org/10.5281/zenodo.1234567)
```

Opción B: WordPress

CMS con interfaz gráfica (GUI)

- **WordPress** es un **CMS** (Content Management System).
- Permite crear y administrar un sitio web mediante una **interfaz gráfica (GUI)** desde el navegador.
- Está pensado para que usuarios no técnicos puedan publicar contenido sin escribir código.

Características principales

- Sitios web **dinámicos** (generalmente con base de datos).
- Contenido editable desde la GUI (entradas, páginas, menús).
- Extensiones mediante **temas y plugins**.

WordPress en un contexto académico

¿Cuándo es una buena opción para un laboratorio?

- Adecuado cuando se necesita:
 - difusión institucional (noticias, comunicados),
 - múltiples editores con roles (admin/editor),
 - formularios (contacto, registro, encuestas),
 - gestión de contenido desde la **GUI**.
- Gran ecosistema de **plugins**.

¿Qué es un plugin?

Un **plugin** es un módulo de software adicional que se instala en WordPress para **extender sus funcionalidades** sin modificar el núcleo del sistema.

Ejemplos de plugins en un laboratorio

- bibliotecas digitales y gestores de PDFs,
- gestores de publicaciones científicas,
- formularios y sistemas de contacto,

Limitaciones de WordPress

Repositorio científico y sostenibilidad

- Requiere **hosting** y **dominio** con costo anual.
- Necesita **mantenimiento constante**:
 - actualizaciones del núcleo,
 - plugins,
 - temas (compatibilidad y seguridad).
- Es un sistema **dinámico** (servidor + base de datos), lo que aumenta la superficie de ataque.

Punto crítico para ciencia

Comparado con GitHub Pages + Markdown (texto plano + Git), WordPress suele ofrecer **menor trazabilidad reproducible** del contenido y mayor dependencia de infraestructura.

Opción C: Wix

Plataforma No-code con interfaz gráfica (GUI)

- **Wix** es una plataforma de creación de sitios web basada en el enfoque **No-code**.
- Permite construir páginas mediante una **interfaz gráfica (GUI)** de tipo arrastrar y soltar (drag-and-drop).
- Todo el proceso se realiza desde el navegador.

¿Qué significa No-code?

El enfoque **No-code** permite crear aplicaciones o sitios web sin escribir código, delegando toda la lógica técnica a la plataforma proveedora.

Ventajas de Wix

Cuándo puede ser una buena opción

- Muy rápido para lanzar un sitio web visualmente atractivo.
- Plantillas modernas listas para usarse.
- No requiere:
 - terminal,
 - Git,
 - conocimientos técnicos de desarrollo web.
- Adecuado para:
 - páginas informativas,
 - divulgación,
 - proyectos personales o comerciales pequeños.

Fortaleza principal

Permite obtener resultados visuales inmediatos con una curva de aprendizaje prácticamente nula.

Limitaciones de Wix

Sostenibilidad y enfoque científico

- Dependencia total de una **suscripción de pago**.
- El sitio y su contenido están ligados a la plataforma.
- No ofrece:
 - control de versiones (Git),
 - trazabilidad de cambios,
 - flujos reproducibles de trabajo.

Riesgo principal para un laboratorio

Si se deja de pagar la suscripción:

- el sitio puede perder funciones,
- mostrar anuncios,
- o quedar inaccesible.

Opción D: GitLab Pages

Sitios estáticos con automatización

- **GitLab Pages** permite publicar **sitios web estáticos** desde un repositorio.
- Funciona de manera similar a **GitHub Pages**, pero con mayor control del proceso.
- Está orientado a proyectos que requieren **automatización explícita**.

El sitio web se genera y se publica automáticamente cada vez que se actualiza el repositorio.

Automatización con CI/CD

Conceptos

¿Qué significa CI y CD?

- **CI — Integración continua:** revisa automáticamente que los archivos y la estructura del sitio estén correctos cada vez que hay cambios.
- **CD — Despliegue continuo:** publica el sitio web automáticamente una vez que la revisión fue exitosa.

¿Qué es un pipeline?

Un **pipeline** es una **secuencia automática de pasos** que:

- revisa los archivos,
- construye el sitio web,
- y lo publica sin intervención manual.

Opción E: Servidor institucional

Infraestructura propia del centro o universidad

- El sitio web se aloja en un **servidor propio** administrado por la institución.
- Es una solución tradicional usada en universidades y centros de investigación.
- Permite alojar:
 - sitios web,
 - bases de datos,
 - archivos grandes,
 - aplicaciones internas.

La institución controla completamente dónde vive el sitio y cómo se administra.

Opción F: Repositorios científicos

Zenodo, OSF y publicación académica

- El contenido principal del laboratorio se aloja en **repositorios científicos especializados**.
- Ejemplos:
 - **Zenodo** (datos, software, DOI),
 - **OSF** (proyectos, datasets, preprints).
- El sitio web actúa solo como **punto de entrada e índice**.

La información científica vive donde debe vivir: en plataformas diseñadas para ciencia.

Flujo de trabajo científico

Publicación, datos y reproducibilidad

- Los artículos, datos y código:
 - se suben a repositorios científicos,
 - reciben identificadores persistentes (DOI),
 - quedan citables y versionados.
- El sitio web del laboratorio:
 - enlaza a esos recursos,
 - organiza por proyectos,
 - no almacena datos pesados.

Ecosistema completo del laboratorio

Cómo encajan todas las piezas

- **Repositorio (Git):** versiones, historia, reproducibilidad.
- **Sitio web:** índice, documentación, visibilidad.
- **Repositorios científicos:** datos, DOI, citación.
- **Automatización (opcional):** despliegue confiable.

Idea central

El sitio web no es el fin, es la **interfaz pública del trabajo científico**.

Comparación de opciones

Opción	Costo	Reproducibilidad	Mantenimiento
GitHub Pages	Bajo	Alta	Bajo
WordPress	Medio	Media	Alto
Wix	Alto	Baja	Bajo
GitLab Pages	Bajo	Muy alta	Medio
Servidor propio	Alto	Alta	Muy alto
Repositorios científicos	Bajo	Máxima	Bajo

Lectura rápida

A mayor control técnico, mayor reproducibilidad; a mayor comodidad gráfica, mayor dependencia.

¿Cómo interpretar la comparación?

Qué significan realmente estos criterios

La tabla resume decisiones que en la práctica afectan el día a día del laboratorio.

Qué significa cada criterio

- **Costo:** no solo dinero, sino dependencia de pagos o infraestructura.
- **Reproducibilidad:** qué tan fácil es rastrear cambios, versiones y resultados a lo largo del tiempo.
- **Mantenimiento:** cuánto esfuerzo humano se requiere para que el sitio siga funcionando correctamente.

Idea clave

En ciencia, el problema no es crear el sitio, sino **mantenerlo usable y confiable durante años.**

¿Qué opción se adapta mejor?

EEG, modelación y experimentos futuros

Para un laboratorio que trabaja con:

- datos experimentales (EEG),
- código y modelos,
- reportes, tesis y artículos,

es clave separar **contenido científico** de **presentación web**.

Lectura práctica de las opciones

- **Wix:** rápido para mostrar, poco adecuado para ciencia.
- **WordPress:** útil para difusión, pero costoso de mantener.
- **Servidor propio:** robusto, pero exige personal técnico continuo.
- **GitHub / GitLab Pages:** pensados para documentación, código y trabajo reproducible.
- **Repositorios científicos:** el lugar correcto para datos y DOI.

Pensando a futuro

Llegada de un desarrollador web al laboratorio

Si en el futuro se desea:

- integrar visualizaciones de EEG,
- crear buscadores de datos,
- desarrollar interfaces más avanzadas,

conviene que la base del sitio sea **estándar y abierta**.

Opción más favorable a largo plazo

GitHub Pages + Jekyll (con Zenodo/OSF como respaldo de datos):

- usa tecnologías web estándar (HTML, CSS, JavaScript),
- todo está versionado con Git,
- un desarrollador puede intervenir sin rehacer el sitio.

Esta opción no cierra puertas: permite empezar simple y crecer sin cambiar de plataforma.

Prueba de concepto

Lecciones aprendidas (caso real - pruebas piloto con **GitHub Pages**)

En un piloto con GitHub Pages + Jekyll se observó:

- errores 404 cuando una carpeta no contiene `index.md`
- enlaces rotos por diferencias de mayúsculas/minúsculas (case sensitive)

Convención recomendada

- carpetas/archivos en minúsculas (`final_project/`)
- cada sección navegable tiene `index.md`
- enlaces con `{{ 'ruta' | relative_url }}`

Más control implica más estructura

YAML, HTML y CSS en la práctica

Al avanzar en el uso de GitHub Pages + Jekyll, se observa que no todo es solo escribir Markdown.

Herramientas que entran en juego

- **Markdown:** contenido académico (textos, listas, enlaces).
- **YAML:** configuración y metadatos (títulos, menús, navegación).
- **HTML + CSS:** control visual y estructura fina del sitio.

Lección del piloto

Estas herramientas dan **mucho más poder y flexibilidad**, pero requieren dedicar más atención y tiempo al inicio.

Apoyo para un producto mínimo viable

IA generativa y terminales inteligentes

Actualmente, el desarrollo web básico no depende únicamente del conocimiento técnico profundo.

Herramientas de apoyo

- **IA generativa (ej. ChatGPT):** ayuda a generar ejemplos de YAML, HTML y CSS, corregir errores y explicar estructuras.
- **Terminales modernas (ej. WARP):** facilitan el uso de Git con autocompletado, historial visual y sugerencias.

Resultado práctico

Aunque no se alcance el nivel de un desarrollador web profesional, es posible construir un **producto mínimo viable (MVP)**:

- funcional,
- ordenado,
- suficiente para el uso académico inicial.

Prueba piloto real

Repositorio académico convertido en sitio web

Se desarrolló una prueba piloto utilizando **GitHub Pages** con un repositorio real llamado `9th-semester`.

Contenido del repositorio

- Documentos académicos alojados directamente en GitHub.
- Organización por carpetas temáticas (materias, proyectos).
- Uso de archivos `README.md` e `index.md` para estructurar la navegación.

El repositorio dejó de ser solo un contenedor de archivos y se convirtió en un **sitio web navegable**.

Cómo se construyó el sitio

De repositorio a web pública

El proceso seguido fue incremental y reproducible:

Pasos realizados

- Creación del repositorio en GitHub.
- Inclusión de un archivo `README.md` como punto de entrada.
- Creación progresiva de archivos `index.md` en cada sección.
- Activación de **GitHub Pages**.

Jekyll y temas

- GitHub Pages ejecuta **Jekyll** automáticamente.
- Se utilizó un tema predefinido (ej. **Cayman**).
- La configuración básica se realizó con un archivo `_config.yml`.

Resultado obtenido

Producto mínimo viable (MVP)

Con herramientas gratuitas y estándares abiertos, se obtuvo un **producto mínimo viable**.

Características del MVP

- Sitio web público y estable.
- Navegación clara entre documentos.
- Historial completo de cambios (Git).
- Compatible con LaTeX, PDFs y texto académico.

Este MVP demuestra que, con Markdown, GitHub Pages, Jekyll y apoyo de herramientas modernas, un laboratorio puede tener presencia web funcional **sin depender de desarrollo web profesional**.

Caso real: piloto con GitHub Pages (MVP)

Repositorio 9th_semester en fase beta

- Se implementó un sitio funcional a partir de un repositorio en GitHub:
 - documentos (PDFs), notas, secciones por curso,
 - navegación por `index.md` en carpetas,
 - contenido escrito principalmente en **Markdown**.
- El sitio está en **desarrollo continuo** (fase beta), pero ya muestra el **potencial** del enfoque.

Demo pública (beta)

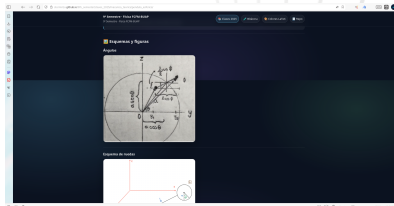
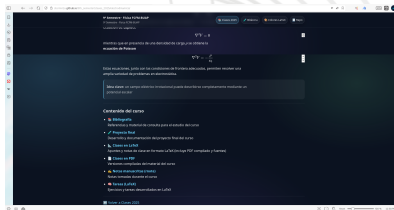
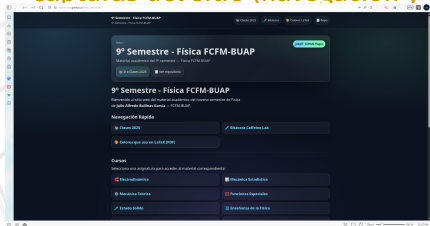
https://stormrizo.github.io/9th_semester/

Idea clave

Con un repositorio bien organizado + GitHub Pages, se obtiene un **sitio web académico navegable** sin infraestructura compleja (servidor propio, base de datos, etc.)

Evidencia visual del piloto

Capturas del sitio (navegación y contenido)



Inicio y lista de cursos

Sección de curso y contenido renderizado

Reports,
tes, e

Nota

El sitio puede mostrar: texto, ecuaciones, listas, links a PDFs, e incluso

Cómo se construyó el MVP

De repositorio a sitio web (flujo simple)

- **Repositorio en GitHub** con carpetas por secciones.
- **README.md** como portada del repositorio (documentación interna).
- `index.md` en cada carpeta navegable:
 - evita errores 404,
 - crea estructura tipo “menú” y navegación.
- **Jekyll** + tema (ej. Cayman u otros):
 - un archivo `_config.yml` define tema, título, rutas, etc.
 - el **layout** es opcional: se puede empezar simple e iterar.

Herramientas recomendadas (opcionales, pero ayudan mucho)

- **VS Code**: editar `.md`, `.yaml`, `.html`, `.css` con orden y vista de proyecto.
- **Terminal (WARP o clásica)**: Git, commits, push, y diagnóstico.
- **IA generativa (ej. ChatGPT)**: acelerar plantillas, índices, fixes de enlaces y estructura.

Conclusiones generales

Lecciones del análisis y del piloto

- Un sitio web de laboratorio no es solo diseño: es **infraestructura para conocimiento científico**.
- El uso de un lenguaje común (**Markdown, Git, terminal**) permite que físicos, matemáticos y colaboradores compartan y preserven información de forma reproducible.
- No todas las plataformas están pensadas para ciencia:
 - mayor comodidad gráfica suele implicar menor control y menor trazabilidad,
 - mayor control técnico implica mayor reproducibilidad y sostenibilidad.

Resultado clave del piloto

El caso real con **GitHub Pages + Jekyll** demuestra que es posible construir un **producto mínimo viable (MVP)** funcional, navegable y escalable sin infraestructura compleja.

Conclusión técnica

Elección más adecuada para el laboratorio

- Para un laboratorio de modelación matemática, neurociencia computacional (EEG) y experimentos futuros, la opción más equilibrada es:

GitHub Pages + Jekyll + Repositorios científicos (opción F)

- Esta combinación ofrece:
 - **Costo cero** en hosting,
 - compatibilidad natural con **LaTeX, PDFs, código y datasets**,
 - **versionado, trazabilidad y reproducibilidad**,
 - facilidad de crecimiento si en el futuro se integra un desarrollador web.
- Herramientas auxiliares como **VS Code, WARP e IA generativa** reducen la barrera técnica y permiten a no especialistas mantener el sistema.

Siguientes pasos recomendados

- Definir la estructura oficial del repositorio del laboratorio.
- Migrar documentos clave (artículos, tesis, reportes).

Referencias